Cloud Computing (CS3081) Mini Project Report

Knowledge Distillation & Deployment of a Command-Parsing Model on Kubernetes

Student: Thirumalasetty Karthik | Roll Number: 230101007 | Repository: https://github.com/SudeepKarthik30/distilled-command-parser-k8s | Release Tag: v1.0

1. Task 1: Object Storage and Parallel Teacher Labelling

Cluster & Storage Setup: A local single-node Kubernetes cluster was instantiated using k3d from cluster/k3d.yaml. SeaweedFS was deployed in namespace data backed by a 5 GiB PersistentVolumeClaim. Automated bootstrapping via scripts/bootstrap.sh provisioned namespaces (app, data, pipeline), generated runtime S3 access credentials stored in Secrets (s3-creds, seaweed-s3), initialized S3 buckets datasets and models, and uploaded the command pool.

Parallel Teacher Sharding: The 3,000 raw input commands from commands_pool.jsonl were partitioned across parallel workers using modulo indexing: line_number % SHARD_COUNT == SHARD_INDEX. Two workers (teacher-worker-shard0 and teacher-worker-shard1) ran concurrently in namespace pipeline as Kubernetes batch Jobs (manifested in k8s/worker-job.yaml, evidence in results/pipeline_workers.txt). Worker 0 processed 1,500 even-indexed queries writing to results/teacher/shard-0.jsonl, and Worker 1 processed 1,500 odd-indexed queries writing to results/teacher/shard-1.jsonl with zero gaps or duplicate queries.

Dataset Cleaning Audit: The cleaning pipeline (worker/clean.py) evaluated all 3,000 raw responses against schema.json, filtering out syntax errors, unsupported intents, and slot hallucinations.

Dataset Audit Category	Evaluation Criteria	Records Filtered	Action Taken
(a) Malformed JSON	Syntax truncation or missing curly braces	4	Rejected
(b) Invalid Key Schema	Keys not strictly {intent, date, time, place}	7	Rejected
(c) Out-of-Domain Intent	Intent not in the 8 permitted categories	0	Rejected
(d) Hallucinated Slots	Extracted slot span not found verbatim in input query	81	Rejected
(e) Duplicate Commands	Repeated queries; lowest command ID retained	48	De-duplicated
Verified Cleaned Records	**Passed all validation filters into train.jsonl**	**2,860**	**Preserved**

Audit equation strictly balances: Input (3,000) = 4 + 7 + 0 + 81 + 48 + 2,860 = 3,000. Verified SHA-256 digest recorded in train.sha256.

2. Task 2: Training the Student (Sequence-Level Knowledge Distillation)

Model Training Setup: The Pythia-70M model was fine-tuned on CPU on the first 1,500 records of train.jsonl. Formatting used prompt masking: Command: {text}\nJSON: (tokens masked with -100 so loss is computed solely on generated tokens). Hyperparameters in training/config.yaml: seed=1007, lr=0.0001, batch_size=8, epochs=3, max_len=128. Training executed via make train and verified with make train SUBSET=50.

Model Architecture	JSON Valid	Intent	Date	Time	Place	Full Frame
Untrained Pythia-70M	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
Teacher (Phi-3.5)	82.0%	59.0%	65.0%	76.0%	71.0%	39.0%
Student v1 (Deployed)	99.0%	74.0%	85.0%	74.0%	61.0%	29.0%
Student v2 (Greedy)	98.0%	68.0%	90.0%	98.0%	65.0%	35.0%

Conceptual Distillation Analysis (Mandatory Explanations):

(a) What sequence-level distillation trains on: Sequence-level distillation (Kim & Rush, 2016) trains the student model on discrete output sequences generated by the teacher model. The teacher's decoded textual output serves as direct hard ground-truth target tokens, trained using standard autoregressive cross-entropy loss without requiring teacher probability distributions.

(b) What logit-based distillation trains on: Logit-based distillation trains the student to match the full continuous token probability distributions of the teacher model. It computes the Kullback-Leibler (KL) divergence between the softened softmax logits of the teacher and the student across the entire vocabulary, transferring dark knowledge about relative token likelihoods.

(c) Why logit-based distillation is not possible here: Logit-based distillation requires identical vocabulary spaces between teacher and student. Phi-3.5 uses a 32,064-token tokenizer, whereas Pythia-70M uses the 50,304-token GPT-NeoX tokenizer. Their vocabularies, token IDs, and subword segmentations do not align, making direct logit comparison mathematically undefined without an external projection matrix. Additionally, Ollama exposes only discrete text generations, not full-vocabulary logit tensors.

(d) Real limitation of our approach: Sequence-level distillation blindly copies teacher errors: when Phi-3.5 hallucinates or outputs invalid slot boundaries (as seen in the 81 rejected records), the student treats those errors as valid targets. Furthermore, Pythia-70M has limited parameter capacity, restricting generalization on complex, multi-clause voice commands.

3. Task 3: The Inference Service in a Container

Web Service Implementation: Built using FastAPI (service/app.py) running Uvicorn on port 8000. Implements GET /healthz (200 OK while alive), GET /readyz (503 Service Unavailable while model is downloading/initializing, 200 OK once model weights are in memory), and POST /parse. Headers X-Model-Version and X-Served-By are attached to every response. Configuration is driven exclusively by MODEL_URI, MODEL_VERSION, S3_ENDPOINT, MAX_NEW_TOKENS, and FAULT_INJECT. Model weights are retrieved from SeaweedFS S3 using credentials loaded from Secret s3-creds.

Validation & Fault Injection: Incoming text queries are validated via Pydantic CommandRequest. Model predictions are validated against schema.json. When FAULT_INJECT=1, the output string is replaced with 'not json', and the service returns HTTP 500 with exact payload {"error": "invalid_model_output"}.

Container Constraints & Peak Memory Evidence: Dockerfile uses python:3.11-slim and CPU-only PyTorch wheels (image size: 1.38 GB, well under the 2.0 GB ceiling). Fifty sequential requests were dispatched to a container clamped under 1 CPU core and 1 GiB RAM (results/docker_50req.txt). All 50 completed with HTTP 200, zero OOM errors (OOMKilled=false), and measured peak memory consumption of 468.2 MiB (45.72% of ceiling).

4. Task 4: Kubernetes Deployment, Scaling, and Reliability

Deployment & Ingress: Deployed in namespace app with 2 initial replicas. Liveness probe targets /healthz and readiness probe targets /readyz. Traefik Ingress routes /parse traffic directly to Service parser. As documented in results/endpoints_during_startup.txt, the Service maintains <none> ready endpoints during container startup until weights finish loading from SeaweedFS.

Autoscaling (HPA): HorizontalPodAutoscaler (k8s/hpa.yaml) targets 70% average CPU (min 1, max 2 replicas). Under sustained Locust load, CPU utilization reached 195%, prompting scale-up from 1 to 2 replicas within 45 seconds. Following load cessation, cooldown stabilization (300 s window) smoothly scaled the Deployment back down to 1 replica (evidenced in results/hpa_watch.txt).

Reliability, Recovery & Rolling Update: Under sustained load, one Pod was deleted to simulate sudden node/container failure; Kubernetes restored the replica within 17.5 seconds, with 450 requests incurring only 4 transient 500 errors (0.89% error rate, strictly below the 2.0% threshold). During rolling update from v1 to v2, 500 requests yielded only 2 errors (0.40% failure rate, below the 0.5% limit), and kubectl rollout undo restored v1 seamlessly.

Cold Start & Load Balancing: Pod cold start from creation to Ready state averaged a median of 7.1 s across 3 consecutive trials (runs: 8.2 s, 6.8 s, 7.1 s, evidence in results/cold_start_logs_run*.txt). Under standard dual-replica Locust load, traffic balanced evenly: 172 requests to Pod A (50.6%) and 168 requests to Pod B (49.4%), comfortably exceeding the 30% per-replica requirement.

Local Cluster Limitation: Single-node k3d runs within a shared container runtime on one physical laptop. While it exercises Kubernetes API controllers, scheduling, and pod lifecycle recovery, it cannot provide true hardware fault tolerance against physical node power failure or hypervisor crash.

5. Task 5: Subnetting and Network Access Control

VLSM Allocation Strategy (10.20.0.0/16): Subnets were allocated in descending order of usable host capacity: app (1,000 hosts needed -> /22 prefix, 1,022 usable hosts, 10.20.0.0 - 10.20.3.255); public (250 hosts needed -> /24 prefix, 254 usable hosts, 10.20.4.0 - 10.20.4.255); data (100 hosts needed -> /25 prefix, 126 usable hosts, 10.20.5.0 - 10.20.5.127); management (20 hosts needed -> /27 prefix, 30 usable hosts, 10.20.5.128 - 10.20.5.159). Full working in network/subnets.md.

Stateless NACL Configuration: Specified in network/nacl.json with custom rule numbering (inbound deny 50 for 198.51.100.0/24, allow 100 for HTTP 80, allow 150 for HTTPS 443, allow 200 for SSH from management, allow 250 for app reply traffic; outbound allow 50 to app port 8000, allow 100 for client replies on ephemeral ports 1024-65535). Verified 100% correct (6/6 packets) via python tools/nacl_sim.py network/nacl.json network/packets_sample.jsonl --check.

Stateless Mechanics & Rule Ordering: Unlike stateful firewalls, a stateless NACL evaluates every packet independently without session tracking. Consequently, reverse return traffic must be explicitly permitted in both directions. Rule evaluation evaluates in strictly ascending numerical order; placing the deny rule for 198.51.100.0/24 at rule 50 ensures malicious traffic is discarded before matching the broader web allow rules at 100 and 150.

6. Task 6: Reproducibility, Testing, and CI/CD

Single-Command Operations: The entire environment builds, provisions storage, imports images, and deploys services from a fresh clone via make up. Clean cluster teardown executes via make down.

Continuous Integration Pipeline (ci.yml): GitHub Actions pipeline executes 5 sequential stages: (1) code-linting with ruff; (2) unit-testing running 4 tests in tests/test_basic.py; (3) security-secret-scan with Gitleaks; (4) model-evaluation downloading model_v1.zip from GitHub Release v1.0 and running predict.py; and (5) quality-gate running score.py --min-validity 95.

Quality Gate Failure Verification: Triggering the pipeline with use_base: true runs inference with the untrained Pythia-70M baseline, producing 0% valid JSON and successfully triggering gate failure (exit status 1), as evidenced in workflow run: https://github.com/SudeepKarthik30/distilled-command-parser-k8s/actions/runs/1918805

7. Task 7: Performance, Cost, and Review

Benchmark Results (Locust 4 users, 2 min, 2 replicas): 342 total requests, 2 failures (0.58% error rate, well under 1% limit). Median latency p50 = 1,250 ms, p95 = 2,700 ms (under 5,000 ms ceiling), throughput = 2.85 req/s. CPU load: 1928m and 1915m per pod. Memory: 742 MiB and 758 MiB. Load balanced: 172 / 168 requests (50.6% / 49.4%). Full metrics in results/loadtest_stats.csv and results/loadtest_headers.json.

Cost Estimation Arithmetic: Each replica is billed for 2 vCPU limit (2 x 3.50 = 7.00 INR/h) and 1 GiB memory limit (1 x 0.45 = 0.45 INR/h), totaling 7.45 INR/replica/h. Two replicas cost 14.90 INR/h. At 2.85 req/s (10,260 req/h), unit economic cost is: (14.90 INR / 10,260 req) * 1,000 = 1.45 INR per 1,000 requests.

SLO Formulation: Objective defined as >= 99% successful requests over one load-test run (342 requests). Permitted error budget = 342 x 0.01 = 3.42 requests. Actual failures observed = 2 requests (58.5% of budget used, SLO fully met at 99.42% success).

Comprehensive Architecture Review Table (With Measured Evidence):

Component or Concern	Evidence (Measured Number)	Identified Limitation	Proposed Improvement
Data & Storage	7 files downloaded over cluster network at startup (median 7.1 s cold start)	Single-node SeaweedFS without high-availability or volume caching	Deploy replicated multi-node MinIO/Ceph; attach PVC node caches
Model Quality	2 of 342 outputs rejected as invalid JSON (0.58% error rate)	Compact 70M student exhibits occasional slot boundary errors	Incorporate larger distillation pool and grammar-constrained decoding
Latency & Throughput	p50 = 1,250 ms, p95 = 2,700 ms, 2.85 req/s on 2 replicas	CPU-bound inference; thread contention occurs if unconstrained	Apply INT8 ONNX Runtime quantization and continuous dynamic batching
Resource Utilization	CPU peaked at 1928m of 2000m limit; RAM at 758 MiB of 1 GiB	Pod capacity saturated near 100% under only 4 concurrent users	Right-size resource limits, scale horizontally across multi-node cluster
Reliability & Recovery	Pod failover restored Ready state in 17.5 s with only 4 errors (0.89%)	Single-node k3d cluster lacks true machine-level fault tolerance	Multi-node Kubernetes topology with PodDisruptionBudgets & anti-affinity
Security & Governance	S3 credentials in Secret s3-creds; unit cost is 1.45 INR / 1k reqs	Kubernetes Secret is only base64; Ingress lacks TLS encryption	Integrate HashiCorp Vault / External Secrets Operator and cert-manager TLS

8. Record of AI Tool Utilization

Antigravity (Gemini) and Anthropic Claude were utilized for architecture design, refactoring, Kubernetes manifests, and diagnosing PyTorch thread contention under CPU limits. Every suggested shell command, manifest, and Python refactor was executed locally and rigorously verified against automated testing suites. The stateless NACL was tested and verified with tools/nacl_sim.py (6/6 passing, 100%). Model scoring was audited with tools/score.py, and request timings were measured directly via Locust. AI inaccuracies (such as default Makefile target precedence and premature readiness probe pass) were identified, corrected, and documented. Distillation labels were produced exclusively by Phi-3.5 via Ollama without synthetic manual fabrication. Full verification logs are detailed in AI_USE.md.